

Desempeño de modelos

Jorge Gallego

Inter-American Development Bank

Bogotá Summer School in Economics — 2026

Agenda de la sesión

- Ya sabemos entrenar varios modelos. Ahora: ¿cómo saber si un modelo es **bueno** y cómo **mejorarlo**?

1. Evaluación del desempeño (métricas y validación)

2. Mejorando el desempeño:

- ▶ Ingeniería y selección de variables; importancia
- ▶ Desbalance de clase
- ▶ Sobreajuste (*overfitting*), ensamblaje y boosting

Parte 1

Evaluación del desempeño

Introducción

- Queremos que los algoritmos pronostiquen **bien** en datos futuros
- Pero hay distintas concepciones (métricas) de lo que es un buen pronóstico
- Veremos varias medidas de desempeño y cuándo conviene cada una
- Y técnicas para estimar el desempeño **futuro** de un modelo

Precisión total... y su trampa

- Lo más obvio: la **precisión total** (proporción de casos bien clasificados)
- Pero cuidado. Supongamos que 1 de cada 1000 bebés nace con un defecto genético
- Un algoritmo que clasifica a **todos** como sanos acierta el 99.9% de las veces
- ¡Pero se equivoca en el 100% de los casos que importan! La precisión total, sola, puede engañar

Matriz de confusión

- **Verdaderos positivos (TP)**: bien clasificados en la categoría de interés
- **Verdaderos negativos (TN)**: bien clasificados fuera de ella
- **Falsos positivos (FP)**: clasificados en ella por error
- **Falsos negativos (FN)**: dejados fuera por error

La matriz de confusión

		Predicción	
		Positivo	Negativo
Real	Positivo	TP	FN
	Negativo	FP	TN

- La diagonal (TP y TN) son los aciertos; fuera de ella, los errores
- Casi todas las métricas de clasificación se derivan de estas cuatro cantidades

Ejemplo: predicción de SMS spam

		Predicted to be Spam	
		no	yes
Actually Spam	no	TN True Negative	FP False Positive
	yes	FN False Negative	TP True Positive

Fuente: Lantz (2015)

Medidas básicas

De la matriz de confusión:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Error\ Rate = 1 - Accuracy$$

Buscamos maximizar la precisión y minimizar el error... pero ya vimos que eso no basta cuando hay **desbalance**.

Estadístico Kappa

- Ajusta la precisión por los aciertos que ocurrirían **por puro azar**

$$\kappa = \frac{Pr(a) - Pr(e)}{1 - Pr(e)}$$

donde $Pr(a)$ es la concordancia observada y $Pr(e)$ la esperada por azar.

- Útil con **desbalance de clase** (donde es fácil tener precisión alta clasificando todo en la clase mayoritaria)
- Va de 0 a 1: ≤ 0.2 muy baja; 0.2–0.4 baja; 0.4–0.6 moderada; 0.6–0.8 alta; > 0.8 muy alta

Kappa: ejemplo (SMS spam)

sms_results\$actual_type	sms_results\$predict_type		Row Total
	ham	spam	
ham	1203	4	1207
	16.128	127.580	0.868
	0.997	0.003	
	0.975	0.026	
	0.865	0.003	
spam	31	152	183
	106.377	841.470	0.132
	0.169	0.831	
	0.025	0.974	
	0.022	0.109	
Column Total	1234	156	1390

Fuente: Lantz (2015)

$$Pr(a) = \frac{1203 + 152}{1390} = 0.974$$

$$Pr(e) = \frac{1207}{1390} \cdot \frac{1234}{1390} + \frac{183}{1390} \cdot \frac{156}{1390} = 0.786$$

$$\kappa = \frac{0.974 - 0.786}{1 - 0.786} = 0.879$$

Concordancia muy alta: los pronósticos van mucho más allá del azar.

Sensibilidad y especificidad

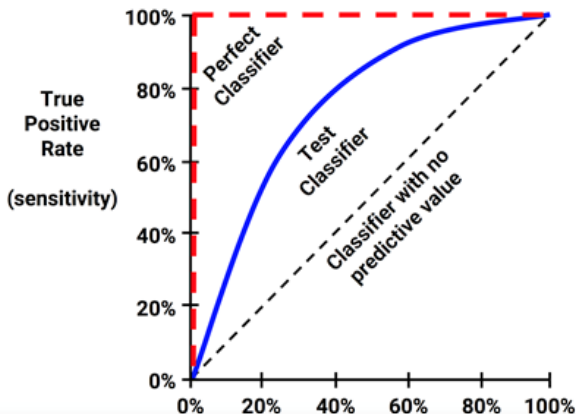
- Capturan un *trade-off*: clasificadores muy agresivos vs. muy conservadores

$$\text{sensibilidad} = \frac{TP}{TP + FN}, \quad \text{especificidad} = \frac{TN}{TN + FP}$$

- **Sensibilidad**: fracción de positivos que detectamos bien
- **Especificidad**: fracción de negativos que detectamos bien
- Un filtro de spam agresivo (alta sensibilidad) bloquea correos buenos (baja especificidad). El punto ideal depende del problema

Curvas ROC

- El *trade-off* sensibilidad–especificidad se ve gráficamente con la **curva ROC**
- Eje y: verdaderos positivos. Eje x: falsos positivos

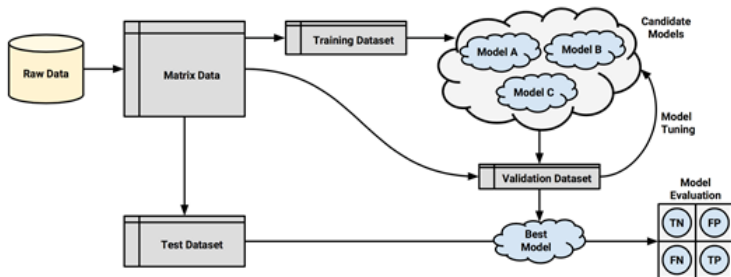


Área bajo la curva (AUC)

- El **AUC** resume la curva ROC en un número
- Clasificador inútil (diagonal de 45°): $AUC = 0.5$
- Clasificador perfecto: $AUC = 1$
- Los reales caen entre 0.5 y 1: cuanto mayor, mejor

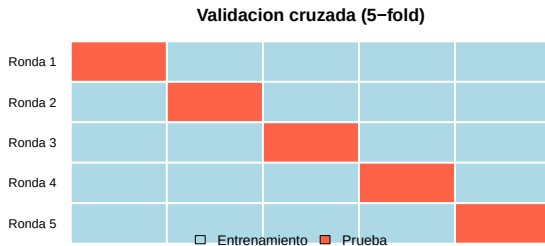
Estimar el desempeño futuro

- Ya usamos el **método holdout**: dividir los datos en entrenamiento y prueba
- Mejor aún: tres partes — entrenamiento, **validación** y prueba
 - ▶ el modelo se afina con validación
 - ▶ y se evalúa, una sola vez, con prueba



Validación cruzada (cross-validation)

- El holdout simple puede dar muestras poco representativas (sobre todo con pocos datos)
- La **validación cruzada k -fold** ($k = 10$ típico): cada grupo se usa una vez como prueba, entrenando con los otros $k - 1$; se promedian las k medidas



Validación cruzada (cross-validation)

- Es más robusta que un solo holdout: usa **todos** los datos para entrenar y para probar (en rondas distintas)
- Da una estimación más estable del desempeño futuro
- Es también la base para elegir **hiperparámetros** (como el λ de Ridge/Lasso, o el k de kNN)
- Costo: hay que entrenar el modelo k veces

Buena práctica: no contaminar la prueba

- La base de **prueba** debe usarse **una sola vez**, al final
- Si elegimos el modelo mirando su desempeño en la prueba, ya la “contaminamos”: la estimación será demasiado optimista
- Por eso afinamos con **validación** (o validación cruzada) y reservamos la prueba para el veredicto final
- Cuidado con la **fuga de información** (*data leakage*): escalar o imputar usando *toda* la base filtra información de la prueba al entrenamiento

Parte 2

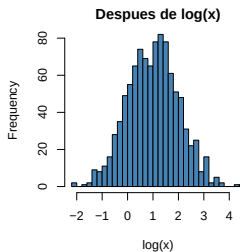
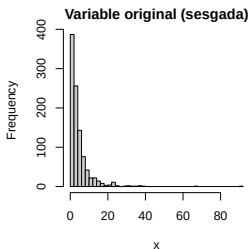
Mejorando el desempeño

Las variables importan más que el modelo

- A menudo, mejores **variables** mejoran más la predicción que un modelo más sofisticado
- *Garbage in, garbage out*: con malas variables, ningún algoritmo salva el día
- Tres ideas relacionadas:
 - ▶ **Feature extraction**: derivar nuevas variables (de texto, imágenes, o reducción de dimensión)
 - ▶ **Feature selection**: quedarse con un subconjunto de variables relevantes
 - ▶ **Feature engineering**: transformar o combinar variables con conocimiento del dominio

Ingeniería de variables: transformaciones

- Transformaciones típicas que ayudan al modelo:
 - ▶ **log** de variables muy sesgadas (ingresos, gastos, precios)
 - ▶ **binning**: discretizar una variable continua en categorías
 - ▶ **dummies** para variables nominales; **interacciones** entre variables
 - ▶ de una **fecha**: día de la semana, mes, festivo; de **texto**: conteos de palabras
- Ya lo hicimos en regresión: edad^2 y la interacción $\text{obesidad} \times \text{tabaquismo}$



Extracción de variables

- A veces no seleccionamos ni transformamos, sino que **construimos** variables nuevas y más compactas
- **Reducción de dimensión** (PCA): resumir muchas variables correlacionadas en unas pocas “componentes” (lo veremos en la próxima sesión)
- **Texto**: convertir documentos en variables (conteos de palabras, *embeddings*)
- **Señales o imágenes**: extraer características relevantes (frecuencias, bordes, formas)

Escalamiento de variables

- Muchos algoritmos son sensibles a la **escala** de las variables (kNN, SVM, regularización)
- Dos formas de rescalar (ya vistas en kNN):

$$\text{min-max: } \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1], \quad \text{z-score: } \frac{x - \mu}{\sigma}$$

- Sin escalar, una variable con valores grandes domina solo por su unidad de medida
- Los árboles, en cambio, son **invariantes** a la escala

¿Por qué seleccionar variables?

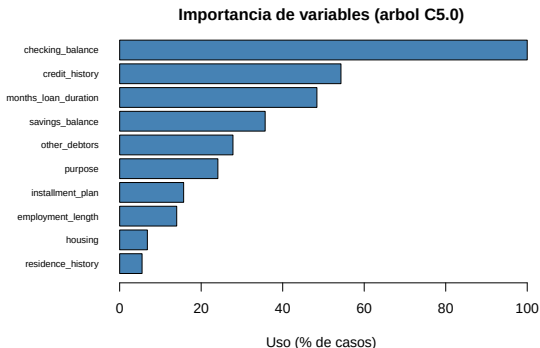
- **Interpretabilidad:** un modelo con pocas variables es más fácil de explicar
- **Menos sobreajuste:** variables irrelevantes agregan ruido
- **Maldición de la dimensionalidad:** con muchas variables frente a pocas observaciones, el espacio se vuelve “vacío” y todo parece lejano
- Lo vimos con Ridge/Lasso: cuando p es grande frente a n , conviene reducir

Métodos de selección de variables

- **Filtro:** rankear cada variable por una medida estadística, *antes* y *aparte* del modelo
 - ▶ correlación con el *outcome*, ganancia de información, chi-cuadrado
- **Envoltura** (*wrapper*): probar subconjuntos de variables y quedarse con el que mejor predice
 - ▶ *forward/backward stepwise*, eliminación recursiva (RFE)
 - ▶ más preciso, pero costoso (entrena muchos modelos)
- **Embebido:** la selección ocurre *dentro* del modelo (Lasso, árboles)

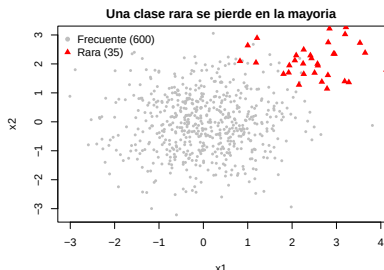
Importancia de variables

- ¿Cuánto aporta cada variable? Dos ideas:
 - ▶ **En árboles/RF:** la reducción promedio de impureza (Gini o entropía) al usar cada variable
 - ▶ **Permutación:** barajar una variable; si la precisión cae mucho, era importante



Desbalance de clase: el problema

- Cuando la clase de interés es **rara** (fraude, una enfermedad, el cliente que compra)
- La precisión total **engaña**: predecir siempre la clase mayoritaria ya da precisión altísima
- El modelo “ignora” la minoría, que suele ser la que importa



Cuando la precisión engaña: mejores métricas

- Con desbalance, en vez de la precisión total miramos:

$$precision = \frac{TP}{TP + FP}, \quad recall = sensibilidad = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

- **Precision:** de lo que predigo positivo, ¿cuánto acierto?
- **Recall:** de los positivos reales, ¿cuántos detecto?
- **F1** combina ambos (media armónica). También: kappa y AUC

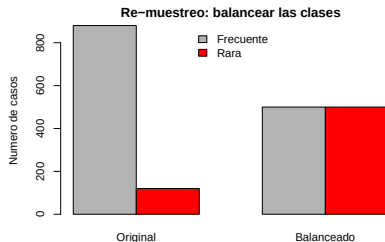
ROC vs. curva precision-recall

- Con desbalance fuerte, la curva ROC puede verse “demasiado optimista”
- Esto porque la especificidad se mantiene alta cuando los negativos son muchísimos
- La **curva precision-recall** se concentra en la clase rara y suele ser más informativa en esos casos
- Regla práctica: con desbalance fuerte, reporta **recall**, **F1** y la **curva precision-recall**, no solo el AUC-ROC

Desbalance: soluciones por re-muestreo

- **Submuestreo:** quitar casos de la mayoría
- **Sobremuestreo:** replicar casos de la minoría
- **Datos sintéticos** (SMOTE, ROSE): generar minoría *nueva*

Se balancea **solo el entrenamiento**; la prueba se deja intacta.



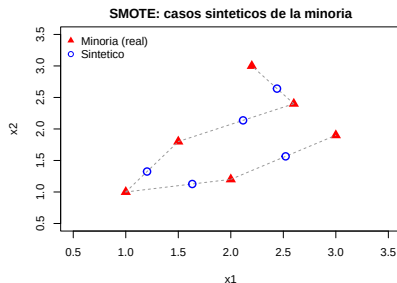
SMOTE: datos sintéticos

- En vez de copiar, SMOTE **interpola** entre un caso minoritario y un vecino cercano:

$$x_{nuevo} = x_i + \lambda (x_{vecino} - x_i)$$

con $\lambda \sim U(0, 1)$

- Crea variedad en la minoría, no solo duplicados



Desbalance: costos y umbral

- Otra vía sin tocar los datos:
- **Costos asimétricos:** decirle al modelo que un falso negativo cuesta más que un falso positivo (o al revés)
- **Mover el umbral:** en vez de clasificar como positivo si $P > 0.5$, bajar el umbral para detectar más positivos
- Todo es el mismo *trade-off*: más sensibilidad a cambio de más falsos positivos

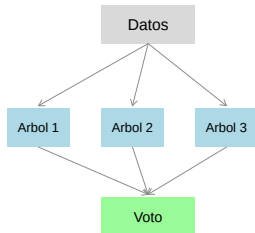
¿Cómo combatir el sobreajuste?

- Conseguir **más datos** (cuando se puede)
- **Validación cruzada** para elegir la complejidad adecuada
- **Regularización** (Ridge, Lasso): penalizar la complejidad
- **Poda** de árboles; limitar su profundidad
- **Ensamblaje**: combinar muchos modelos

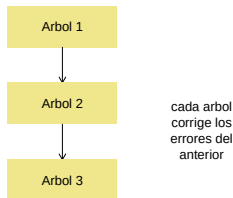
Ensamblaje: combinar modelos

- **Ensamblar** = combinar muchos modelos para predecir mejor que uno solo

Bagging (paralelo)



Boosting (secuencial)



- **Bagging:** árboles en paralelo (reduce varianza). **Boosting:** árboles en secuencia (reduce sesgo)

Bagging (repaso) y Random Forests

- **Bagging**: entrenar árboles sobre muestras *bootstrap* y **promediar/votar**
- Reduce la **varianza** de modelos inestables (como los árboles)
- **Random Forests**: además decorrelaciona los árboles usando un subconjunto aleatorio de variables en cada nodo (lo vimos en S2)
- Bagging y RF se entrenan **en paralelo**: el orden no importa

Boosting

- Entrena modelos “débiles” (árboles pequeños) **en secuencia**
- Cada modelo da **más peso** a los casos que el anterior clasificó **mal**
- La predicción final es una combinación ponderada de todos
- Ya lo usamos sin nombrarlo: el parámetro `trial`s de C5.0 hace *boosting*

AdaBoost, gradient boosting y XGBoost

- **AdaBoost**: sube el peso de los casos mal clasificados en cada ronda
- **Gradient boosting**: cada nuevo árbol ajusta los **errores (residuos)** del conjunto anterior
- **XGBoost** (y LightGBM): implementaciones muy eficientes y regularizadas
- Dominan muchas competencias de predicción con datos tabulares

¿Bagging o boosting?

- **Bagging / RF**: atacan la **varianza**. Buenos con modelos inestables; difíciles de sobreajustar; fáciles de paralelizar
- **Boosting**: ataca el **sesgo**. Suele dar mayor precisión, pero es más sensible al ruido y a los hiperparámetros (puede sobreajustar)
- **Al taller**: tres talleres — evaluación (ROC, CV), selección de variables, y desbalance + boosting

Resumen de la sesión

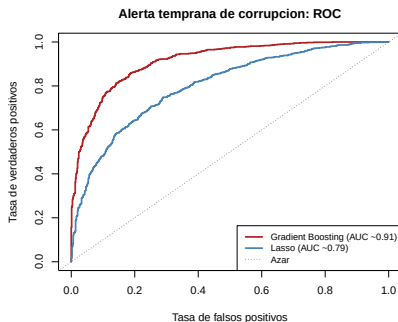
- La **precisión total** no basta: con desbalance, mira kappa, sensibilidad/especificidad, F1 y AUC
- Estima el desempeño **futuro** con holdout y **validación cruzada** (sin contaminar la prueba)
- Mejora el modelo con mejores **variables** (ingeniería, extracción, selección, importancia)
- Maneja el **desbalance** (re-muestreo, SMOTE/ROSE, costos)
- Controla el **sobreajuste** (sesgo-varianza, CV, regularización) con **ensamblaje** y **boosting**

Aplicación real en Colombia: alerta temprana de corrupción

- ¿Se puede **anticipar** la corrupción en la contratación pública, en vez de solo castigarla después?
- Un estudio con datos de **SECOP** (más de 2 millones de contratos) construye un modelo de **alerta temprana**
- Predice tres desenlaces: investigación de la **Contraloría**, incumplimiento del contrato e ineficiencias

Alerta temprana: desempeño y desbalance

- Usa **Lasso** y **gradient boosting**; los casos positivos son **raros** (clase muy desbalanceada)
- Como lo que importa es **priorizar**, se evalúa con ROC/AUC y métricas de *ranking* (MAP@k, NDCG)



Alerta temprana: resultados y lección

- El **gradient boosting** supera al Lasso (AUC entre **0.79** y **0.91**)
- Permite ordenar los contratos por riesgo y concentrar la vigilancia donde más importa
- Caso de manual para esta sesión: **desbalance**, **ensamblajes** (*boosting*) y **métricas** más allá de la precisión
- *Fuente:* Gallego, Rivero y Martínez (2021), "Preventing rather than punishing: An early warning model of malfeasance in public procurement", *International Journal of Forecasting*, 37(1).